



History of JavaScript

JavaScript is one of the most widely-used programming languages in web development. It was created to make web pages interactive and dynamic.

Year	Event
1995	JavaScript was created by Brendan Eich at Netscape
1996	Microsoft released a similar language called JScript for Internet Explorer
1997	JavaScript was standardized as ECMAScript (ES1) by ECMA International
2009	Node.js introduced -- allows JS to run on the server
2015	Major update with ES6 (ECMAScript 2015) -- added let, const, arrow functions, classes, etc.
Today	JavaScript is used for web apps, mobile apps, server-side apps, game dev, AI, etc.



JavaScript Founder

	Name	Brendan Eich
	Company	Netscape Communications
	Created in	1995
	Later Roles	Co-founded Mozilla, Firefox, Brave



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

? Why the Name “JavaScript”?

Originally called Mocha, then LiveScript

Renamed to JavaScript to ride the popularity of Java at the time

Despite the name, Java and JavaScript are very different

🔑 JavaScript Today

Runs in all modern browsers (Chrome, Firefox, Edge, Safari, etc.)

Used in Frontend (React, Vue, Angular) and Backend (Node.js)

Supported by frameworks, libraries, tools, and a huge community



Internal JavaScript

Internal JavaScript means writing JavaScript inside the HTML file using the `<script>` tag, typically within the `<head>` or `<body>` section.

✓ Syntax Example

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal JavaScript Example</title>

  <!-- Internal JavaScript in the <head> -->
  <script>
    function greet() {
      alert("Hello from internal JavaScript!");
    }
  </script>
</head>
<body>

  <button onclick="greet()">Click Me</button>

</body>
</html>
```



◆ Key Points

```
Placed in <script>    Code is enclosed inside <script> ... </script>
Written in HTML      No external file needed
Executes in-browser   Runs as soon as the page loads (or on event)
Easy for small scripts Great for learning and small demos
```

! Tip:

For larger projects, it's better to use external JavaScript (`<script src="script.js"></script>`) to keep your code organized.

External JavaScript

External JavaScript means writing JavaScript code in a separate `.js` file and linking it to your HTML file using the `<script src="...">` tag.

✓ Why Use External JavaScript?

```
Reusability    Use the same script in multiple pages
Clean HTML     Keeps HTML and JavaScript code separate
Easier to Maintain  Modify script in one file, changes everywhere
```



Better Organization Helpful **in** large projects

♦ 1. Create HTML File (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>External JavaScript Example</title>
  <script src="script.js" defer></script> <!-- Link external JS -->
</head>
<body>
  <button onclick="sayHello()">Click Me</button>
</body>
</html>
```

♦ 2. Create JavaScript File ([script.js](#))

```
function sayHello() {
  alert("Hello from external JavaScript!");
}
```

When to Use

Use Internal JS => Small examples, testing , One-time scripts



Use External JS=>Projects, production websites , Repeated or large-scale code

Javascript Boxes

 **JavaScript Dialog Boxes: `alert()`, `confirm()`, and `prompt()`**

JavaScript provides three basic dialog boxes to interact with users:

① `alert()` Box

Used to show a simple message to the user.

```
alert("This is an alert!");
```

♦ Behavior:

- Displays a message with an OK button.
- No return value (i.e., undefined).

② `confirm()` Box

Used to ask the user for confirmation (Yes/No or OK/Cancel).

```
let result = confirm("Are you sure?");
```

♦ Behavior:

- Shows OK and Cancel buttons.
- Returns:
 - **true** if user clicks OK
 - **false** if user clicks Cancel

```
if (confirm("Do you want to delete this?")) {  
    console.log("Deleted");  
} else {  
    console.log("Cancelled");  
}
```

3 prompt() Box

Used to ask the user for input.

```
let name = prompt("What is your name?");
```

♦ Behavior:

- Shows a text input with OK and Cancel

- Returns:
 - The user input (as a string) if OK is clicked
 - `null` if Cancel is clicked

```
let name = prompt("Enter your name:");
if (name !== null) {
  alert("Hello, " + name);
}
```

Note:

- These dialogs pause script execution until the user responds.
- They are synchronous (blocking)
- Their appearance is controlled by the browser (not styled with CSS).

Let , var & const

These are the **three ways** to declare variables in JavaScript. Here's how they differ:

✓ 1. **var** — The Old Way (before ES6)

✓ Function-scoped

⚠ Gets **hoisted** (moved to top) but initialized as `undefined`

- ✗ Can be **redeclared** and **updated**
- ✗ Not block-scoped (can cause bugs)

```
if (true) {  
  var x = 10;  
}  
console.log(x); // 10 -- still accessible outside the block
```

✓ 2. **let** — Modern Variable (ES6+)

- ✓ Block-scoped (inside {})
- ✓ Can be **updated**, ✗ but **not redeclared** in same scope
- ✓ Hoisted, but ✗ **not initialized** — using it before definition gives an error

```
if (true) {  
  let y = 20;  
}  
console.log(y); // ✗ Error: y is not defined
```

✓ 3. **const** — For Constants (ES6+)

- ✓ Block-scoped



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

✗ Cannot be **redeclared** or **updated**

✓ Must be **initialized at the time of declaration**

```
const person = { name: "Alice" };  
person.name = "Bob"; // ✓ Allowed  
person = { name: "Charlie" }; // ✗ Error
```

✓ Best Practice:

- ✓ Use **let** for values that will change.
- ✓ Use **const** by default.
- ✗ Avoid **var** unless supporting very old browsers.

Scope in Javascript

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

Scope defines **where variables can be accessed or referenced** in your code. It determines the **lifetime and visibility** of variables.

✚ 3 Main Types of Scope:

✓ 1. Global Scope

- Variables declared **outside** of any function or block.
- Accessible **anywhere** in your code (except inside **module** scope).

```
let globalVar = "I'm global";  
function show() {  
    console.log(globalVar); // ✓ Accessible  
}
```

✓ 2. Function Scope

- Variables declared with **var** inside a function are **only accessible inside** that function.

```
function greet() {  
    var message = "Hello!";  
    console.log(message); // ✓  
}  
console.log(message); // ✗ Error
```

✓ 3. Block Scope

- Variables declared with **let** or **const** inside **{}** (like **if**, **for**, **while**, etc.) are **not accessible outside** that block.

```
if (true) {  
  let x = 10;  
  const y = 20;  
}  
console.log(x); // ✗ Error
```

Operators in JavaScript

✓ 1. Arithmetic Operators

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	6 / 3	2
%	Modulus (Remainder)	5 % 2	1
**	Exponentiation	2 ** 3	8
++	Increment	a++	a + 1
--	Decrement	a--	a - 1

✓ 2. Assignment Operators

Operator	Description	Example
=	Assign	x = 10
+=	Add and assign	x += 5
-=	Subtract and assign	x -= 3
*=	Multiply and assign	x *= 2
/=	Divide and assign	x /= 4
%=	Modulus and assign	x %= 2

✓ 3. Comparison Operators

==	Equal to (loose)	5 == "5"	true
===	Equal value and type	5 === "5"	false
!=	Not equal (loose)	5 != "5"	false
!==	Not equal (strict)	5 !== "5"	true
>	Greater than	6 > 5	true
<	Less than	3 < 4	true
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	4 <= 4	true

✓ 4. Logical Operators



```
&&    AND    true && false    false
!      NOT    !true    false
```

✓ 5. String Operator

```
let name = "John" + " " + "Doe";
console.log(name); // "John Doe"
```

✓ 6. Ternary Operator (Conditional)

```
let age = 18;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result); // "Adult"
```

✓ 7. Type Operators

```
typeof    Returns data type    typeof "Hello" → "string"
```

For Loop

The **for** loop is used to repeat a block of code a specific number of times. It has three parts:



```
for (let i = 0; i < 5; i++) {  
  console.log("Number is: " + i);  
}
```

Output:

```
Number is: 0  
Number is: 1  
Number is: 2  
Number is: 3  
Number is: 4
```

Explanation of Syntax:

```
let i = 0 → Initialization: runs once at the beginning.  
i < 5 → Condition: loop runs as long as this is true.  
i++ → Increment: updates the loop counter after each iteration.
```

Looping Through an Array:

```
let fruits = ["apple", "banana", "mango"];  
  
for (let i = 0; i < fruits.length; i++) {  
  console.log(fruits[i]);  
}
```



Output

```
apple  
banana  
mango
```

While Loop

The **while** loop repeats a **block of code** as long as a condition **is true**.

Example:

```
let i = 0;
```

```
while (i < 5) {  
    console.log("Number is: " + i);  
    i++;  
}
```

Output:

```
Number is: 0  
Number is: 1  
Number is: 2  
Number is: 3  
Number is: 4
```

Key Points:

- The condition is checked before each iteration.

- If the condition is initially **false**, the loop block will not run at all.
- Make sure to update the loop variable (**i++**) inside the loop, or it could run forever.

Infinite Loop Example (⚠️ Avoid in practice):

```
while (true) {  
    // This will run forever unless there's a `break`  
}
```

If Else

The **if...else** statement is used to run code based on a condition.

```
let age = 18;  
  
if (age >= 18) {  
    console.log("You are an adult.");  
} else {  
    console.log("You are a minor.");  
}
```

Output: You are an adult.

Example 2: `if...else if...else`

```
let marks = 85;

if (marks >= 90) {
  console.log("Grade: A");
} else if (marks >= 75) {
  console.log("Grade: B");
} else if (marks >= 60) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
```

Output: Grade: B

Example 3: `if` without `else`

```
let temperature = 30;

if (temperature > 25) {
  console.log("It's hot outside.");
}
```

Output: It's hot outside.

Ternary Operator

The ternary operator is a short-hand way of writing `if...else`.

- ♦ Syntax: **condition ? expressionIfTrue : expressionIfFalse;**

- ♦ **Example 1:**

```
let age = 18;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result);
Output : Adult
```

- ♦ **Example 2 (Direct inside console.log):**

```
let isLoggedIn = true;
console.log(isLoggedIn ? "Welcome!" : "Please log in.");
Output : Welcome!
```

- ✓ **When to Use:**

- For simple conditions
- To assign values based on a condition

Switch Case

The **switch** statement is used when you want to compare one value against many possible values. It's an alternative to multiple **if...else if** conditions.

- ♦ Important Notes:

break exits the switch block after a match. Without it, **fall-through** will occur.

default is optional but useful when no cases match.

- ♦ Example without **break** (Fall-Through Behavior):

```
let color = "green";

switch (color) {
  case "red":
    console.log("Stop");
  case "yellow":
    console.log("Slow down");
  case "green":
    console.log("Go");
  default:
    console.log("Signal broken");
}
```

Output : Signal broken

Array in Javascript

In JavaScript, arrays are used to store multiple values in a single variable. They are one of the most common data structures in the language.

♦ Creating an Array

```
// Using square brackets (most common)
let fruits = ["Apple", "Banana", "Mango"];
// Using the Array constructor
let numbers = new Array(1, 2, 3, 4, 5);
```

♦ Accessing Array Elements

```
console.log(fruits[0]); // Apple
console.log(fruits[2]); // Mango
```

♦ Modifying Array Elements

```
fruits[1] = "Orange"; // Changes "Banana" to "Orange"
```

✓ Array Method `push()` – Add to **end** of the array

```
let colors = ["Red", "Green"];
colors.push("Blue");
```

```
console.log(colors); // ["Red", "Green", "Blue"]
```

✓ Array Method **pop()** – Remove from the **end** of the array

```
let colors = ["Red", "Green", "Blue"];
let removedColor = colors.pop();

console.log(colors); // ["Red", "Green"]
console.log(removedColor); // "Blue"
```

✓ Array Method **unshift()** – Add to the **beginning** of the array

```
let numbers = [2, 3, 4];
numbers.unshift(1);

console.log(numbers); // [1, 2, 3, 4]
```

✓ Array Method **shift()** – Remove from the **beginning** of the array

```
let numbers = [1, 2, 3, 4];
let removedNumber = numbers.shift();

console.log(numbers); // [2, 3, 4]
console.log(removedNumber); // 1
```

✓ Array Method `splice()` – Used to **add**, **remove**, or **replace** elements in an array.

```
let fruits = ["Apple", "Banana", "Mango"];

// Remove 1 element at index 1
fruits.splice(1, 1);
console.log(fruits); // ["Apple", "Mango"]

// Add 2 elements at index 1
fruits.splice(1, 0, "Orange", "Pineapple");
console.log(fruits); // ["Apple", "Orange", "Pineapple", "Mango"]

// Replace 1 element at index 2
fruits.splice(2, 1, "Kiwi");
console.log(fruits); // ["Apple", "Orange", "Kiwi", "Mango"]
```

✓ Array Method `slice()` – Used to **copy** a portion of the array.

✗ **Does NOT modify** the original array.

```
let fruits = ["Apple", "Orange", "Kiwi", "Mango"];

// Get items from index 1 to 3 (excluding 3)
let sliced = fruits.slice(1, 3);

console.log(sliced); // ["Orange", "Kiwi"]
console.log(fruits); // Original remains unchanged
```

✓ Array Method `join()` – The `join()` method combines **all elements of an array** into a **single string**, with a **separator** of your choice.

```
let fruits = ["Apple", "Banana", "Mango"];
let result = fruits.join();

console.log(result); // "Apple,Banana,Mango"
```

✓ Array Method `indexOf()` – The `indexOf()` method is used to **find the position (index)** of an element in an array.

```
let colors = ["Red", "Green", "Blue", "Green"];
let index = colors.indexOf("Green");

console.log(index); // 1 (first occurrence)
```

1. It returns the **first matching index**.
2. Returns **-1** if the element is **not found**.
3. It's **case-sensitive**:

🔄 `for...of` – Used to **iterate over values** (arrays, strings, etc.)

✓ Example with Array:

```
let colors = ["Red", "Green", "Blue"];

for (let color of colors) {
  console.log(color);
}
// Output: Red, Green, Blue
```

✓ Example with String:

```
for (let char of "Hello") {
  console.log(char);
}
// Output: H, e, l, l, o
```

🕒 **for...in** – Used to **iterate over keys (indexes or property names)**

✓ **Example with Object:**

```
let person = { name: "Alice", age: 25 };

for (let key in person) {
  console.log(key + ": " + person[key]);
}
// Output:
// name: Alice
// age: 25
```

✓ Example with Array:

```
let fruits = ["Apple", "Banana", "Mango"];

for (let index in fruits) {
  console.log(index + " => " + fruits[index]);
}

// Output:
// 0 => Apple
// 1 => Banana
// 2 => Mango
```

! Tip:

- Use **for...of** when working with **arrays or strings**.
- Use **for...in** when looping over **objects or keys**.

✓ Callback in Array Methods (like **forEach**, **map**):

```
let nums = [1, 2, 3];

nums.forEach(function(n) {
```



```
console.log(n * 2);  
});  
// Output: 2, 4, 6
```

Object in Javascript

In JavaScript, an object is a collection of key-value pairs used to store related data and functionality.

✓ Creating an Object **Using Object Literal**

```
let person = {  
  name: "Alice",  
  age: 25,  
  isStudent: false  
};
```

✓ Creating an Object **Using new Object()**

```
let person = new Object();  
person.name = "Alice";  
person.age = 25;
```

✓ Accessing Object Properties



Dot Notation

```
console.log(person.name); // Alice
```

Bracket Notation

```
console.log(person["age"]); // 25
```

Adding / Updating / Deleting Properties

```
person.city = "Mumbai"; // Add  
person.age = 26; // Update  
delete person.isStudent; // Delete
```

Object with Methods (Function inside object)

```
let person = {  
  name: "Alice",  
  greet: function() {  
    console.log("Hello, " + this.name);  
  }  
};  
person.greet(); // Hello, Alice
```

Looping through an Object

```
for (let key in person) {  
  console.log(key + ": " + person[key]);  
}
```

✓ Nested Objects

```
let user = {  
  name: "John",  
  address: {  
    city: "Delhi",  
    zip: 110011  
  }  
};  
  
console.log(user.address.city); // Delhi
```

Summary

- Objects are key-value pairs.
- Can hold any data type, including arrays, other objects, and functions.
- Used for modeling real-world entities (e.g., user, product, car).



Function in Javascript

A function is a block of code designed to perform a specific task. You define it once and can reuse it multiple times.

✓ Why Use Functions?

- Avoid repeating code (DRY principle)
- Break program into smaller tasks
- Reuse logic easily
- Improve readability and maintainability

◆ Function Types in JavaScript

1. Named Function (Function Declaration)

```
function greet() {  
    console.log("Hello, world!");  
}  
greet(); // Output: Hello, world!
```



2. Anonymous Expression

```
let sayHi = function() {  
  console.log("Hi!");  
};  
sayHi(); // Output: Hi!
```

3. Arrow Function (ES6)

```
let add = (a, b) => {  
  return a + b;  
};  
console.log(add(2, 3)); // Output: 5
```

◆ Function with Parameters and Return Value



```
function multiply(a, b) {  
    return a * b;  
}  
let result = multiply(4, 5);  
console.log(result); // Output: 20
```

◆ Default Parameters

```
function greet(name = "Guest") {  
    console.log("Hello, " + name);  
}  
greet(); // Hello, Guest  
greet("Alice"); // Hello, Alice
```

◆ Rest Parameters

```
function myfunc(...numbers) {  
    console.log(numbers)  
}  
console.log(myfunc(1, 2, 3)); // [1,2,3]
```

Callback Function in Javascript

A callback function is a function passed as an argument to another function, and it's usually called later after some operation completes.

📌 "Call me back when you're done!" – that's the idea behind a callback.

◆ Why Use Callbacks?

- To control execution order (especially in async tasks).
- To reuse logic by passing different behavior as arguments.

✓ Basic Example:

```
function greet(name, callback) {  
  console.log("Hello, " + name);  
  callback(); // calling the callback function  
}  
function sayBye() {  
  console.log("Goodbye!");  
}  
  
greet("Alice", sayBye);
```

🔍 Output:

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
Hello, Alice  
Goodbye!
```

✓ Anonymous Callback Example:

```
function processUserInput(callback) {  
  let name = "Bob";  
  callback(name);  
}  
  
processUserInput(function(userName) {  
  console.log("Welcome, " + userName);  
});
```

setTimeout

setTimeout is a JavaScript function used to **delay the execution** of a function or block of code by a specified amount of time (in milliseconds).

```
setTimeout(() => {  
  console.log("This message appears after 2 seconds");  
}, 2000);
```

setInterval



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

`setInterval` is a JavaScript function used to repeatedly execute a function after a fixed time interval (in milliseconds), until it's stopped with `clearInterval`.

```
setInterval(() => {  
    console.log("Runs every 2 seconds");  
}, 2000);
```

DOM In Javascript

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

The DOM (Document Object Model) is a programming interface that represents an HTML document as a tree structure where each node is an object (like elements, attributes, and text).

JavaScript can use the DOM to access, modify, add, or delete elements in an HTML page.

♦ DOM Tree Structure Example:

```
<html>
  <body>
    <h1>Hello</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Document

```
├── html
│   └── body
│       ├── h1
│       └── p
```

✓ Accessing Elements

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56



```
document.getElementById() => Get one element by ID
document.getElementsByClassName() => Get elements by class name
document.getElementsByTagName() => Get elements by tag name
document.getElementsByName() => Get elements by attribute name
document.querySelector() => Get the first matching CSS selector
document.querySelectorAll() => Get all elements matching selector
```

DOM Method getElementById()

`getElementById()` is a DOM method used to **select a single & First HTML element** by its unique `id` attribute.

```
<p id="message">Hello World</p>
<p id="message">Hello World</p>

<button onclick="changeText()">Click Me</button>

function changeText() {
    document.getElementById("message").innerText = "Text Changed!";
}
```

Output of the above code:



```
<p id="message">Text Changed!</p>
<p id="message">Hello World</p>

<button onclick="changeText()">Click Me</button>
```

DOM Method getElementByTagName()

`getElementsByTagName()` is a DOM method used to **select all HTML elements** with a specific tag name (e.g., all `<p>`, `<div>`, `<li>`, etc.).

```
<p>First paragraph</p>
<p>Second paragraph</p>
<p>Third paragraph</p>
<button onclick="highlightParagraphs()">Highlight</button>
```

```
function highlightParagraphs() {
  const paragraphs = document.getElementsByTagName("p");
  paragraphs[0].innerHTML = 1000;
```

```
paragraphs[1].innerHTML = 2000;  
paragraphs[2].innerHTML = 3000;  
  
}
```

In **getElementsByTagName()** we can select multiple elements using the **index number** as given in the above example.

DOM Method getElementByClassName()

getElementsByTagName() is a DOM method used to **select all HTML elements** that share a common **class** name.

```
<p class="para">First paragraph</p>  
<p class="para">Second paragraph</p>  
<p class="para">Third paragraph</p>  
<button onclick="highlightParagraphs()">Highlight</button>
```

```
function highlightParagraphs() {  
    const paragraphs = document.getElementsByClassName("para");
```



```
paragraphs[0].innerHTML = 1000;  
paragraphs[1].innerHTML = 2000;  
paragraphs[2].innerHTML = 3000;  
}
```

In **getElementsByClassName()** we can select multiple elements using the **index number** as given in the above example.

DOM Method getElementByName()

getElementByName() is a DOM method used to select **all HTML elements** with a specific **name** attribute. It is **commonly used with form elements** like **<input>**, **<radio>**, **<checkbox>**, etc.

```
<form>  
  <input type="radio" name="gender" value="Male"> Male  
  <input type="radio" name="gender" value="Female"> Female  
  <input type="radio" name="gender" value="Other"> Other  
  <button type="button" onclick="getGender()">Submit</button>  
</form>
```

```
function getGender() {  
    const genders = document.getElementsByName("gender");  
    for (let i = 0; i < genders.length; i++) {  
        if (genders[i].checked) {  
            alert("Selected gender: " + genders[i].value);  
        }  
    }  
}
```

In **getElementsByName()** we can select multiple elements using the **index number** as given in the above example With for Loop.

DOM Method querySelector()

querySelector() is a powerful and modern DOM method that allows you to select the first element that matches a CSS-style selector.

In **querySelector()** we can select id , class and html tag.

- ♦ Select by ID:
document.querySelector("#title");
- ♦ Select by Class:
document.querySelector(".box");
- ♦ Select by Tag:
document.querySelector("p");
- ♦ Select by Attribute:
document.querySelector('input[type="text"]');

```
<p class="info">This is paragraph 1</p>
<p class="info">This is paragraph 2</p>

let firstInfo = document.querySelector(".info");
firstInfo.innerHTML = "Hello Paragraph";
```

- ◆ Only the **first .info paragraph** content will get changed.

```
<p class="info">Hello Paragraph</p>
<p class="info">This is paragraph 2</p>
```

DOM Method querySelectorAll()

`querySelector()` is a powerful and modern DOM method that allows you to select all elements that match a CSS-style selector.

In `querySelectorAll()` we can select id , class and html tag.

- ◆ Select by ID:
`document.querySelector("#title");`
- ◆ Select by Class:
`document.querySelector(".box");`
- ◆ Select by Tag:
`document.querySelector("p");`
- ◆ Select by Attribute:
`document.querySelector('input[type="text"]');`

```
<p class="info">This is paragraph 1</p>
<p class="info">This is paragraph 2</p>

let firstInfo = document.querySelectorAll(".info");
firstInfo[0].innerHTML = "Hello Paragraph";
firstInfo[1].innerHTML = "Hello Paragraph";
```

- ◆ Here **both the paragraphs values will be changed using index number**

```
<p class="info">Hello Paragraph</p>
<p class="info">Hello Paragraph</p>
```

innerHTML

innerHTML is a DOM property used to **get or set** the **HTML content** inside an element.

```
<div id="info">Hello <u>World</u></div>

let content = document.getElementById("info").innerHTML;
console.log(content); // Output: Hello World
```

Accepts raw HTML strings, not just plain text.

Can **insert or replace entire blocks of HTML**.

innerText

innerText is a DOM property used to get or set the *visible text* inside an HTML element — ignoring any HTML tags.

```
<div id="info">Hello <u>World</u></div>

let content = document.getElementById("info").innerText;
console.log(content); // Output: Hello <u>World</u>
```

Get: Returns only the **rendered, visible text** inside the element.

Set: Replaces all content with plain text (HTML is treated as text).

Event Listener

An event listener in JavaScript is a way to respond to user actions (like clicks, typing, mouse movements, etc.) on a web page by attaching a function to an element.

```
<button id="myBtn">Click Me</button>

document.getElementById("myBtn").addEventListener("click", function() {
    alert("Button was clicked!");
});
```

✓ Removing an Event **Listener**:



```
function sayHi() {  
    alert("Hi!");  
}  
const btn = document.getElementById("myBtn");  
btn.addEventListener("click", sayHi);  
  
// Later remove it:  
btn.removeEventListener("click", sayHi);
```

Event Handling

Event handling is how JavaScript responds to user interactions (like clicks, typing, mouse movements, etc.) on a web page. You use event handlers to define what should happen when an event occurs.

✓ 3 Common Ways to Handle Events:

1. HTML Attribute (Inline)

```
<button onclick="sayHello()">Click Me</button>  
  
<script>  
    function sayHello() {  
        alert("Hello!");  
    }
```

```
}  
</script>
```

2. JavaScript Property With Event Handling

```
let btn = document.getElementById("myBtn");  
btn.onclick = function () {  
    alert("Button clicked!");  
};
```

3. `addEventListener()` (Best Practice)

```
document.getElementById("myBtn").addEventListener("click", function () {  
    alert("Clicked via event listener");  
});
```

`event` Object:

When an event occurs, a special **event object** is passed to the handler function containing information about the event.

```
document.addEventListener("keydown", function(event) {  
    console.log("Key pressed:", event.key);  
});
```

Common Events:

- click => Mouse click
- dblclick => Double click
- mouseover => Mouse enters element
- mouseout => Mouse leaves element
- keydown => Key is pressed down
- keyup => Key is released
- change => Input value changes
- Submit => Form is submitted
- load => Page or image finishes loading

DOM Node Manipulation

createElement() in JavaScript

createElement() is a DOM method used to **create a new HTML element** using JavaScript — dynamically and without writing it in the HTML file.

 Example – Creating and Adding an Element:

```
<div id="container"></div>
let container = document.getElementById("container");
```



```
let heading = document.createElement("h2");
heading.innerText = "Hello from JS!";
heading.style.color = "blue";

container.appendChild(heading);
```

✓ Example – Create a List Dynamically:

```
let fruits = ["Apple", "Banana", "Cherry"];
let ul = document.createElement("ul");

fruits.forEach(fruit => {
  let li = document.createElement("li");
  li.innerText = fruit;
  ul.appendChild(li);
});

document.body.appendChild(ul);
```

✓ append() vs appendChild()

✓ 1. `appendChild()`

- Appends a **single node** (like an element or text node).
- ✗ Cannot append multiple nodes or strings at once.
- ✓ Returns the appended node.
- ✗ Throws an error if you try to append a string.

```
let div = document.createElement("div");
let p = document.createElement("p");
p.innerText = "Hello";
div.appendChild(p);
```

✓ 2. `append()`

- ✓ Can append **multiple nodes and/or strings**.
- ✓ More flexible: accepts elements **and** text.
- ✓ Doesn't return anything.
- ✓ Safer for mixed content (e.g., text + elements).

```
let div = document.createElement("div");
let span = document.createElement("span");
```



```
span.innerText = "Hello";  
div.append("Start ", span, " End"); // Text + element + text
```

✓ Practical Use:

Use `appendChild()` when:

- You're appending a single DOM node.
- You want to get the returned node.

Use `append()` when:

You need to add **multiple nodes or text**.

- You want **simpler and more flexible** code.

DOM Traversal Properties in JavaScript

DOM traversal means navigating the DOM tree — moving between parent, child, and sibling elements using built-in properties.

```
parentNode    Returns the parent node  
parentElement Returns the parent element  
childNodes    All child nodes (elements, text, comments)  
children      Only child elements  
firstChild    First child node (any type)  
firstElementChild First element child
```

```
lastChild      Last child node
lastElementChild  Last element child
nextSibling    Next node (can be text/comment/etc.)
nextElementSibling  Next element sibling
previousSibling  Previous node
previousElementSibling  Previous element sibling
```

Ajax

AJAX stands for Asynchronous JavaScript And XML.

It allows web pages to communicate with servers and update parts of a page without reloading the whole page.

✓ What Can AJAX Do?

- Load data from a server without refreshing the page
- Send data to the server in the background
- Fetch data from APIs (e.g., weather, user info)
- Improve user experience with dynamic content

 AJAX with **XMLHttpRequest** in JavaScript



XMLHttpRequest is the **original way** to make AJAX requests in JavaScript — used to **send and receive data from a server without refreshing the page.**

"AJAX" redirects here. For other uses, see Ajax (disambiguation).

Asynchronous JavaScript and XML

First appeared March 1999

Ajax (also AJAX /ˈeɪdʒæks/; short for "asynchronous JavaScript and XML"[1][2]) is a set of web development techniques that uses various web technologies on the client-side to create asynchronous web applications. With Ajax, web applications can send and retrieve data from a server asynchronously (in the background) without interfering with the display and behaviour of the existing page. By decoupling the data interchange layer from the presentation layer, Ajax allows web pages and, by extension, web applications, to change content dynamically without the need to reload the entire page.[3] In practice, modern implementations commonly utilize JSON instead of XML.

Ajax is not a technology, but rather a programming pattern. HTML and CSS can be used in combination to mark up and style information. The webpage can be modified by JavaScript to dynamically display (and allow the user to interact with) the new information. The built-in XMLHttpRequest object is used to execute Ajax on webpages, allowing websites to load content onto the screen without refreshing

the page. Ajax is not a new technology, nor is it a new language. Instead, it is existing technologies used in a new way.

✓ Sample Code

```
let xhr = new XMLHttpRequest();

// Track the state changes of the request.
xhr.onreadystatechange = function () {
    if (xhr.readyState ==4  && xhr.status==200) {
        // status 200 is a successful return.
        // readyState 4 means the request is done.
        // We will receive data from an api
        console.log(xhr.responseText);
    }
    else if (xhr.readyState ==4  && xhr.status==404) {
        // status 404 is a failure return.
        // We will receive an error
        console.log(xhr.responseText);
    }
};

// define the request
xhr.open('GET', 'https://fakestoreapi.com/products');
// Send the request to send-ajax-data.php
xhr.send(null);
```

readyState in XMLHttpRequest

The **readyState** property of **XMLHttpRequest** defines the **current state of the request**. It helps you **track the progress** of the AJAX call.

readyState Values and Meanings

Value	State	Description
0	UNSENT	Request not initialized
1	OPENED	open() has been called
2	HEADERS_RECEIVED	send() has been called, headers received
3	LOADING	Downloading; response is being received
4	DONE	Request completed; full response received

open() Method in AJAX (XMLHttpRequest)

The **.open()** method in **XMLHttpRequest** is used to initialize a new HTTP request.

send() Method in AJAX (XMLHttpRequest)

The **.send()** method is used to send the HTTP request that was set up using **.open()** in an **XMLHttpRequest** object.

onreadystatechange in AJAX (**XMLHttpRequest**)

The **onreadystatechange** property defines a function to execute every time the **readyState** of an **XMLHttpRequest** object changes.

responseText in AJAX (**XMLHttpRequest**)

The **responseText** property contains the response data as a string from an AJAX request made using **XMLHttpRequest**.

JSON

JSON stands for JavaScript Object Notation.

It is a lightweight data format used for storing and exchanging data, especially between a client and a server.

◆ **JSON Syntax Rules**

- Data is in key/value pairs
- Keys and string values must be in double quotes

- Data is separated by commas
- It is similar to a JavaScript object, but more strict

♦ Example of JSON

```
{  
  "name": "John",  
  "age": 30,  
  "isStudent": false,  
  "skills": ["JavaScript", "HTML", "CSS"]  
}
```

♦ Converting Between JSON and JavaScript

✓ JSON to JavaScript Object

```
let jsonStr = '{"name": "Alice", "age": 25}';  
let obj = JSON.parse(jsonStr); // converts JSON string → JS object  
console.log(obj.name); // Output: Alice
```

✓ JavaScript Object to JSON

```
let jsObj = { name: "Bob", age: 28 };
let jsonString = JSON.stringify(jsObj); // converts JS object → JSON string
console.log(jsonString); // Output: {"name":"Bob","age":28}
```

♦ Use Cases of JSON

- Sending/receiving data from an API
- Configuration files (e.g., `package.json`)
- Storing structured data

Promise

A Promise in JavaScript is an object that represents the eventual completion (or failure) of an **asynchronous** operation and its resulting value.

A Promise has 3 States

✓ States of a Promise:

1. **Pending** – Initial state, neither fulfilled nor rejected.
2. **Fulfilled** – The operation completed successfully.

3. **Rejected** – The operation failed.

```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data loaded!");
  }, 2000);
});

promise.then(result => {
  console.log(result); // Output after 2 sec: Data loaded!
}).catch(error => {
  console.error(error);
}).finally(() => {
  console.error('Promise Completed');
});
```

In JavaScript, when working with **Promises**, you use `.then()`, `.catch()`, and `.finally()` to handle the **results**, **errors**, and **cleanup tasks**, respectively.

✓ `.then()` – Handle Success

Used to handle a **resolved (fulfilled)** Promise.

✓ `.catch()` – Handle Errors

Used to handle a **rejected** Promise (like a network error or thrown exception).

✓ `.finally()` – Run Regardless of Outcome

Used for cleanup or final steps that should run **whether the Promise was fulfilled**



or rejected.

Fetch

The `fetch()` method in JavaScript is used to make HTTP requests (like GET, POST, PUT, DELETE) to servers. It returns a Promise, which resolves to the `Response` object representing the response to the request.

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log('Error:', err));
```

✓ Common Methods:

Method	Description
GET	Retrieve data
POST	Send new data
PUT	Update existing data
DELETE	Delete data

🧠 Tips:

- `fetch()` **won't reject** on HTTP errors (like 404 or 500). You need to check

www.php-training.in / www.phpcourse.in

Mulund : 303,304,309 Shree Samarth Plaza , RRT Road, Opp. Mulund Station . Mulund West . Mumbai 400080

Vile Parle : Datapoint computers: vijaya Apt , Ground Floor ,Bajaj Road , Vile parle West, Vile Parle (West) . MUMBAI -56

`response.ok`.

- To parse the response, use methods like `.json()`, `.text()`, `.blob()` etc.
- For older browsers, use a **polyfill** or fallback to `XMLHttpRequest`.

Async Await

`async` and `await` are modern syntax for handling asynchronous operations, built on top of Promises. They make asynchronous code look and behave more like synchronous code, improving readability.

📌 Example :: Using `fetch` with `async` / `await`

```
async function fetchPost() {
  try {
    const response = await
fetch('https://jsonplaceholder.typicode.com/posts/1');
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Fetch error:', error);
  }
}

fetchPost();
```

Why use **async** / **await**?

- Easier to read than **.then()** chains
- Cleaner error handling
- Better for sequential API calls

Closure

A closure is a function that remembers the variables from its outer (enclosing) scope, even after that outer function has finished executing.

Definition:

A closure is created when a function is defined inside another function, and the inner function accesses variables from the outer function.

Real-World Example: Counter with Closure

```
function createCounter() {  
  let value = 0;  
  
  return {  
    increment() {  
      value++;  
      console.log(value);  
    },  
    decrement() {
```

```
        value--;  
        console.log(value);  
    }  
};  
}  
  
const counter = createCounter();  
counter.increment(); // 1  
counter.increment(); // 2  
counter.decrement(); // 1
```

Why are Closures Useful?

- **Data Privacy:** Variables stay hidden and protected.
- **Stateful Functions:** Keep track of values between function calls.
- **Callbacks and Event Handlers:** Often use closures.

Object Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects**, which can contain **data (properties)** and **code (methods)**.

4 Pillars of OOP:

1. **Encapsulation** – Grouping data & functions inside an object
2. **Abstraction** – Hiding complex details and showing only essentials

- 3. **Inheritance** – One class can inherit properties/methods from another
- 4. **Polymorphism** – One interface, many implementations

1. ES6 Class Syntax (Recommended)

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    console.log(`Hey, I'm ${this.name}`);  
  }  
}  
  
let p2 = new Person("Charlie", 28);  
p2.greet(); // Output: Hey, I'm Charlie
```

♦ Inheritance (Using **extends**)

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  
  speak() {
```

```
        console.log(`${this.name} makes a sound`);
    }
}

class Dog extends Animal {
    speak() {
        console.log(`${this.name} barks`);
    }
}

let dog = new Dog("Rex");
dog.speak(); // Output: Rex barks
```

◆ Encapsulation using # (Private Fields)

```
class BankAccount {
    #balance = 0;

    deposit(amount) {
        if (amount > 0) this.#balance += amount;
    }

    getBalance() {
        return this.#balance;
    }
}

let acc = new BankAccount();
acc.deposit(1000);
```



INFINITECH TRAINING INSTITUTE

[9619 404 202 , 9619 404 225]

[Mulund , Vile Parle]

```
console.log(acc.getBalance()); // Output: 1000
```